# Operating Systems Technologies

(Tecnologias de Sistemas Operativos)

## A Quick OS Recap

University of Minho

2025-2026

# Operating Systems

## Main Design Principles

- **Abstract** the “low-level” details of hardware
  - Virtualizes physical resources, abstracting their complexity and heterogeneity (e.g., hardware architectures, disk models, GPU models)
  - E.g., Linux runs under multiple hardware architectures
- Provide an **interface** (System Call API) for users to tell the OS what to do
  - To run programs: *fork()*, *exec()*, …
  - Manage memory: *brk()*, …
  - Access devices: *open()*, *read()*, *write()*, …

**These design principles provide Convenience to users!**

# Operating Systems

## Main Design Principles

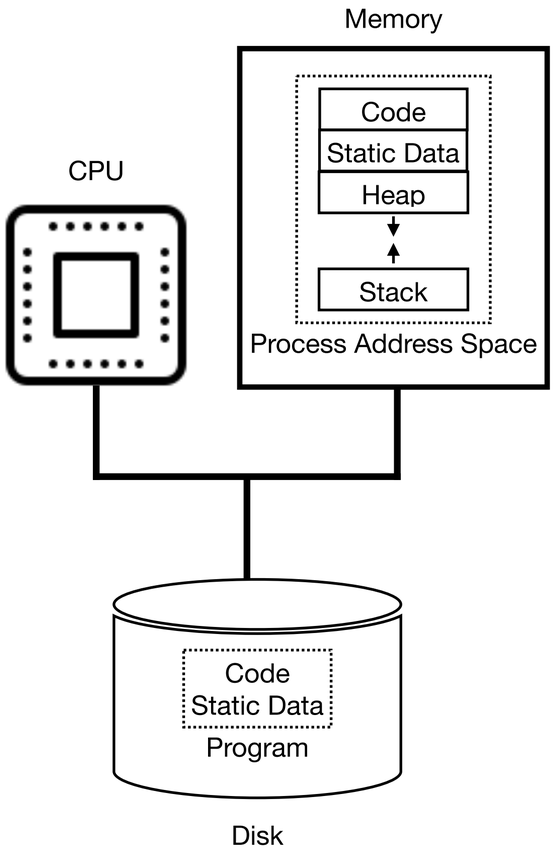
- Ensure **performance** when running multiple programs accessing shared physical resources
  - What program should be scheduled next? What program should write data to disk next?
  - **Important:** the OS must make decisions quickly and without excessive resource usage overhead
- Ensure **protection** by **isolating** running programs from each other and from the OS
  - A malicious/buggy program cannot access private memory regions from other programs (and OS)
  - Also, it should not be able to overtake the CPU and “block” the computer
- Other principles
  - **Reliability:** A failure of the OS means that all applications will fail
  - **Security:** An extension of the protection principle against malicious programs (e.g., ransomware)
  - **Energy efficiency, mobility, …**

**These design principles provide Efficiency to users!**

# Process

## Definition

- Program: passive entity (e.g., an executable file sitting on disk with several instructions and static data)
- Process: (a running program[1]) active entity, executing instructions and accessing system resources (CPU, Memory, Disk)
- The activity of a process can be represented by
  - The value of the **Program Counter** (PC) and other **CPU registers**
    - The PC is a special register identifying the instruction to execute next
  - **Address space**, i.e., the memory layout of a process
    - Where the instructions ("code"), data (variables, structures, …) of a process reside
  - List of **open files**, as processes may do I/O
  - This and other information are stored at the **Process Control Block** (PCB) structure



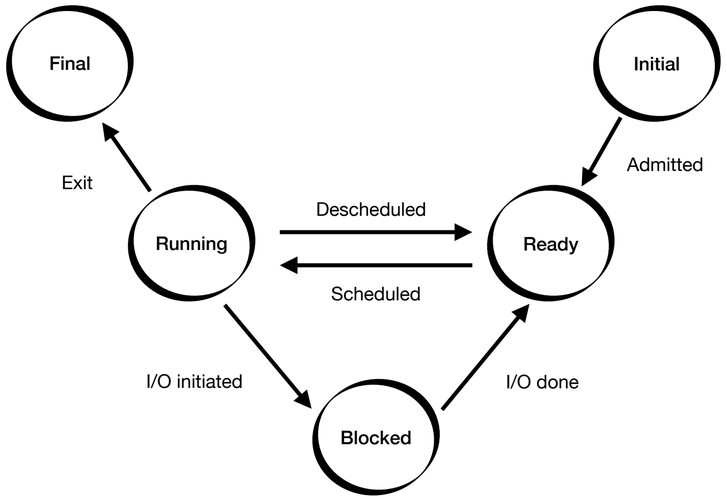

[1] Informal definition, one program can create several processes

# Process

## States

- The process abstraction is fundamental for virtualizing the CPU
  - In a given CPU, the OS can start a process, stop it, and run another one
- With the right mechanisms (**context switch**) and policies (**scheduling algorithms**)
  - Time sharing and multiprogramming are possible!

| Time | $Process_0$ | $Process_1$ | Notes |
|---|---|---|---|
| 1 | Running | Ready | |
| 2 | Running | Ready | |
| 3 | Running | Ready | $Process_0$ initiates I/O |
| 4 | Blocked | Running | $Process_0$ is blocked, so $Process_1$ runs |
| 5 | Blocked | Running | |
| 6 | Blocked | Running | |
| 7 | Ready | Running | I/O is done |
| 8 | Ready | Running | $Process_1$ finishes |
| 9 | Running | Finished | |
| 10 | Running | Finished | $Process_0$ finishes |




[1] Informal definition, one program can create several processes

# Modes of Execution

## Restricted Operations

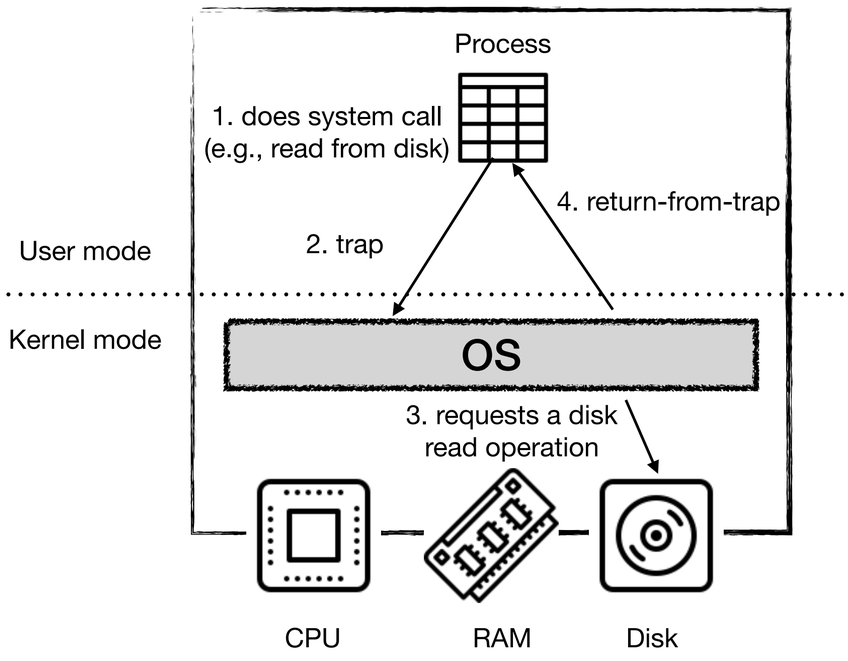
- User processes (*i.e.*, running programs) cannot freely do restricted operations (*e.g.*, I/O requests to disk, gain access to resources such as CPU or memory)
- The OS must mediate this interaction (with the aid of hardware)!
  The CPU provides two modes of execution[1]
  - **User mode:** Code running in the CPU cannot do restricted (privileged) operations. (e.g., a process in user mode cannot directly do disk I/O operations)
  - **Kernel mode:** Code executing in the CPU has full access to the hardware and can do any restricted operation. **The OS runs in this mode!**
- **Issues:** What if a user process attempts a privileged operation?

[1] *A single bit, stored at the processor, indicates which mode the CPU is currently running in*

# System Calls

## Restricted Operations

- **System calls** allow the kernel (OS) to expose privileged operations to user processes (e.g., process creation, communication, memory allocation, I/O)
- To execute a system call, the process calls a special **trap** instruction, raising the privilege level to **kernel mode** and jumping into the OS code (**trap handler**)
- The OS can now perform privileged operations (on behalf of the user process)
- When finished, the OS calls a **return-from-trap** instruction that reduces the privilege level to **user mode** and resumes the execution of the process



# Mode Switching

## Traps and Interrupts

- **Two types of instructions** lead to transferring control from user to kernel mode
- A **trap** is generated by the current process running on a processor
  - The user program asks for an OS service (as we saw before)
    e.g., when the process issues a system call (in UNIX)
  - The user program performs an illegal action (an exception)
    e.g., division by zero, illegal memory access, …
- **Interrupts** are caused by devices and may not be related to the currently running process
  - When a device (disk, keyboard, network card) requires the OS's attention
    e.g., a disk read is ready, there is keyboard input to be read
  - When the timer interrupt is generated by the hardware

**Note:** *Terminology changes depending on the system (x86, xv6). Some call trap to all instructions, other use the terms interchangeably…*

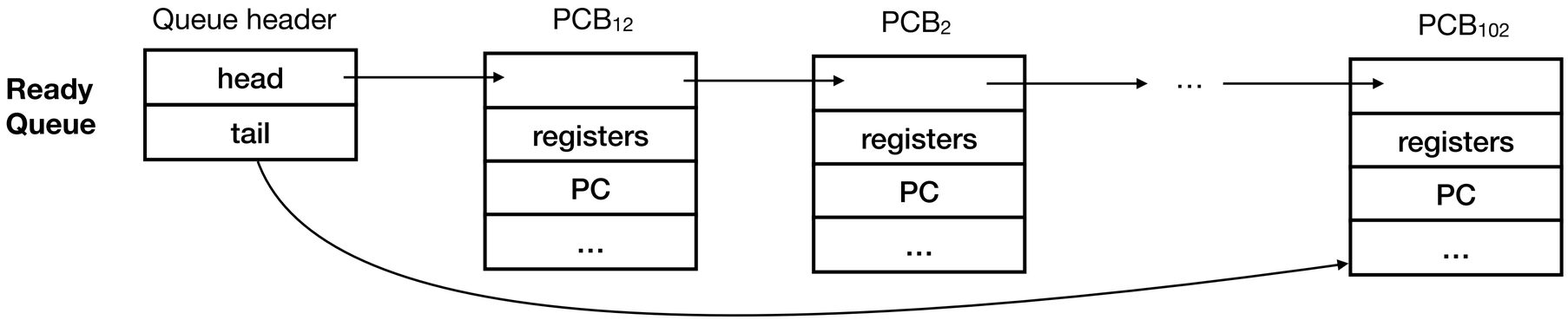
# Saving and Restoring Context

## Context Switching

- When the OS regains control (cooperatively or non-cooperatively)
  - It must decide to keep running the same process or to run another one
  - This decision is made by the OS **scheduler**
- When the scheduler decides to switch $\text{process}_A$ with $\text{process}_B$, a **context switch** is made
  - The OS must save information of $\text{process}_A$ (e.g., program counter, memory limits, …) into its $\text{PCB}_A$ structure
  - Switches contexts, by loading the information kept at $\text{PCB}_B$ and running $\text{process}_B$ at the CPU

# OS Scheduler

- The number of processes ready to run is significantly higher than the number of available CPU cores
- The PCBs of *Ready* processes are kept in a **Ready queue**
- The **OS scheduler** is responsible for choosing the next process to execute
  - But what about the **policies**? What process should be scheduled next?



# Some algorithms

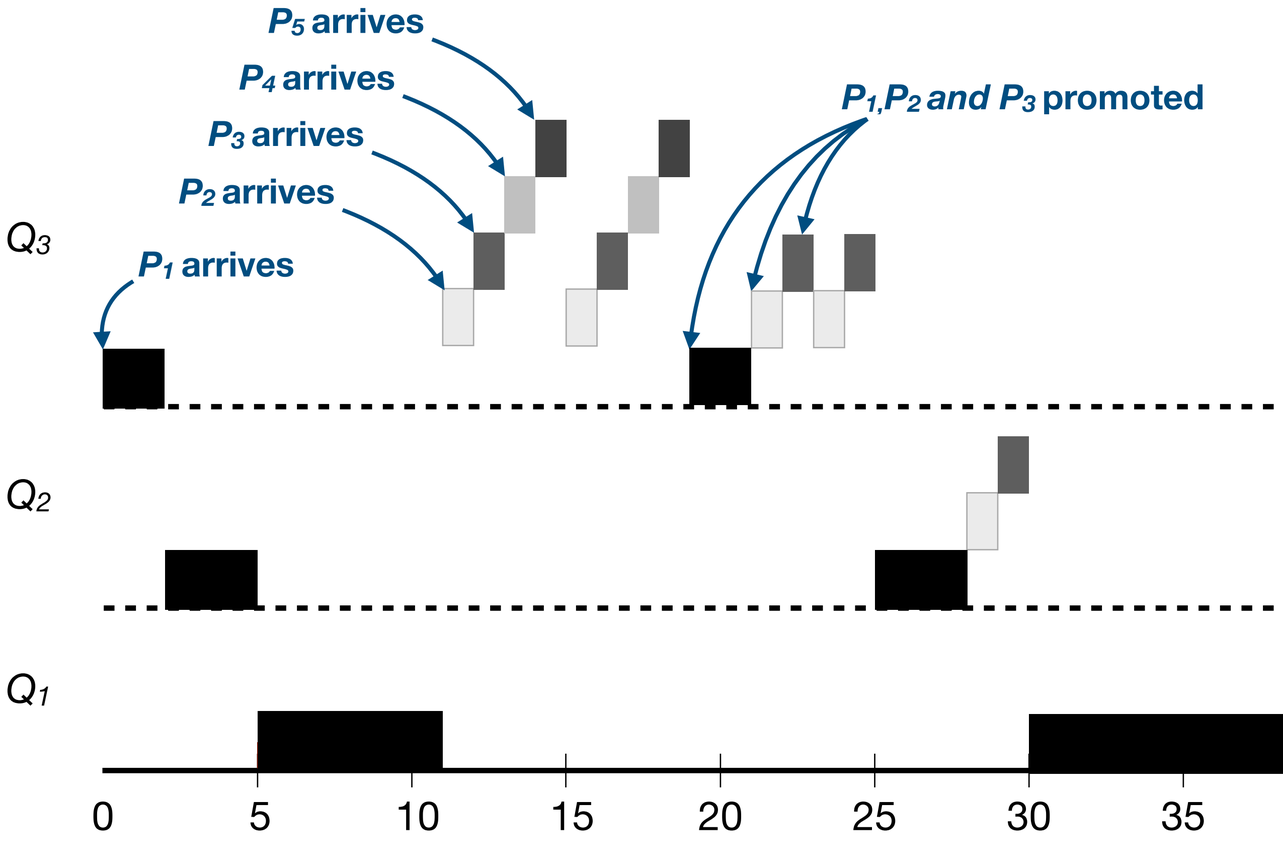
## FCFS, SJF, STCF, RR

- **FCFS (FIFO)** - Schedule the process that arrived first (earlier)
  - Simple and easy to implement!
  - Turnaround time is very sensitive to the order of arrival
- **SJF** and **STCF** - Schedule the process with the shortest (remaining) runtime
  - Both improve turnaround time. STCF is better when processes arrive at distinct times
  - Both require knowing the runtime in advance, and are bad for response time
- **RR** - Run a process for a given time slice (*quantum*), then switch to the next
  - A fair scheduler, good for improving response time (good for interactive systems)
  - Bad algorithm for turnaround time (fair algorithms are bad for this metric)
- What about an algorithm that balances turnaround and response times?

# MLFQ

## Brief Overview

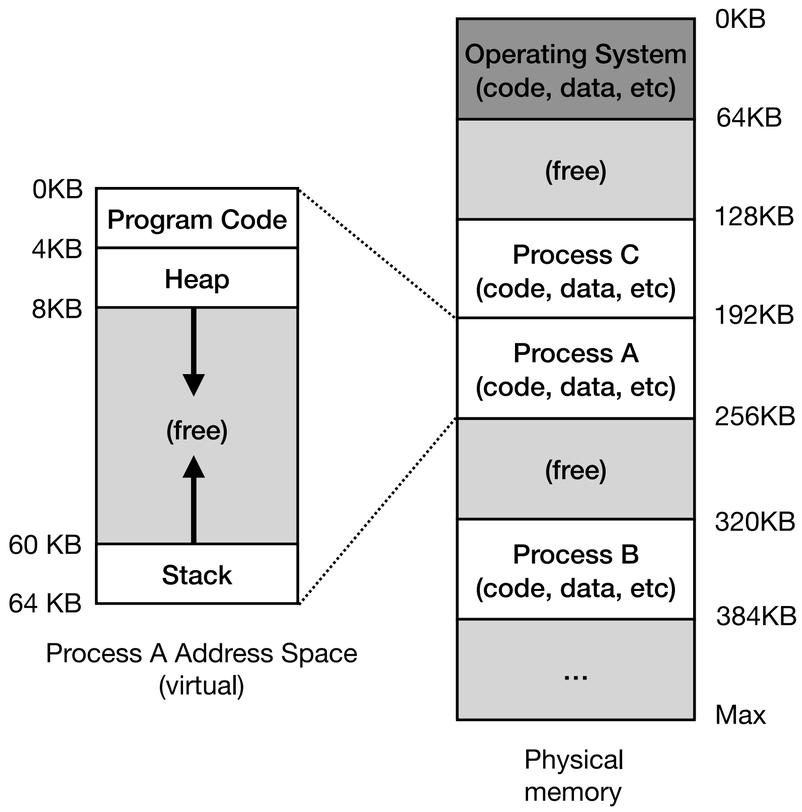
- Short-duration processes run with high priority
  - Good for turnaround time
  - Approximates STCF
- High-priority processes are frequently switched (*allotment*)
  - Good for response time
  - Approximates RR
- Several tuning knobs in MLFQ
  - How many queues?
  - What should be the allotment time?
  - When should the priority boost be called?



# Virtualizing Memory

## Virtual and physical addresses

- Back to the address space… and how it is mapped to memory
- The address space of *Process A* is not really loaded into physical memory at addresses 0 to 64KB
  - When *Process A* accesses address 0 (**virtual address**), the request is mapped to address 192KB at main memory (**physical address**)
- **Note:** Address spaces are much larger in real systems
  - 32 bit ($2^{32}$ = *4GB*) or 64 bits ($2^{64}$ = *16EB*)
  - Even larger than the physical memory at your computer…
- What techniques allow the abstraction of a **private**, **potentially large** address space for **multiple processes**?

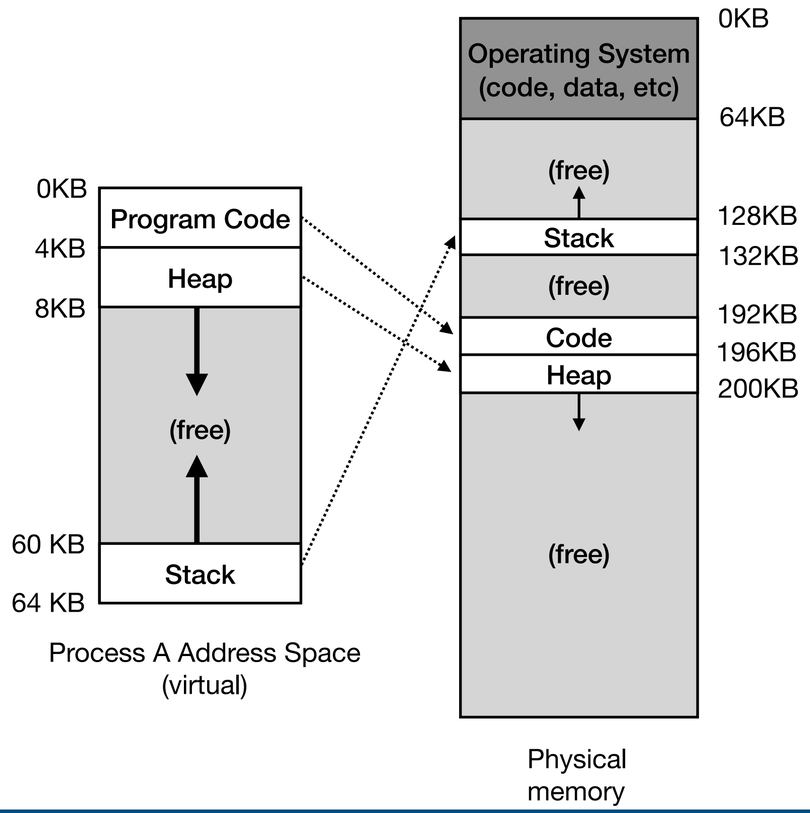


# Segmentation

## Generalized Base/Bounds

| Segment | Base Register | Bounds Register |
|---|---|---|
| CODE | 196 608 (192KB) | 4096 (4KB) |
| HEAP | 200 704 (196KB) | 4096 (4KB) |
| STACK | 135 168 (132KB) | 4096 (4KB) |

- Each logical **segment** of the address space (e.g., code, heap, stack) is placed at a different physical memory region
  - **Segmentation** can be implemented with several base and bounds CPU registers, one per logical segment
- **Context switching** requires saving and restoring all these registers
- Each segment can be **relocated** across memory as the system runs
  - Possible to share segments across multiple processes



# Memory Management

## Challenge with segmentation

- With **segmentation**, the OS chops physical memory into variable-sized chunks
  - Chunks with varied sizes can lead to **external fragmentation**, making memory allocation challenging over time
- How can one avoid this challenge of using segmentation?
  - What **new techniques** are required?
  - How can one make such techniques **efficient** in terms of **performance** and **space**?

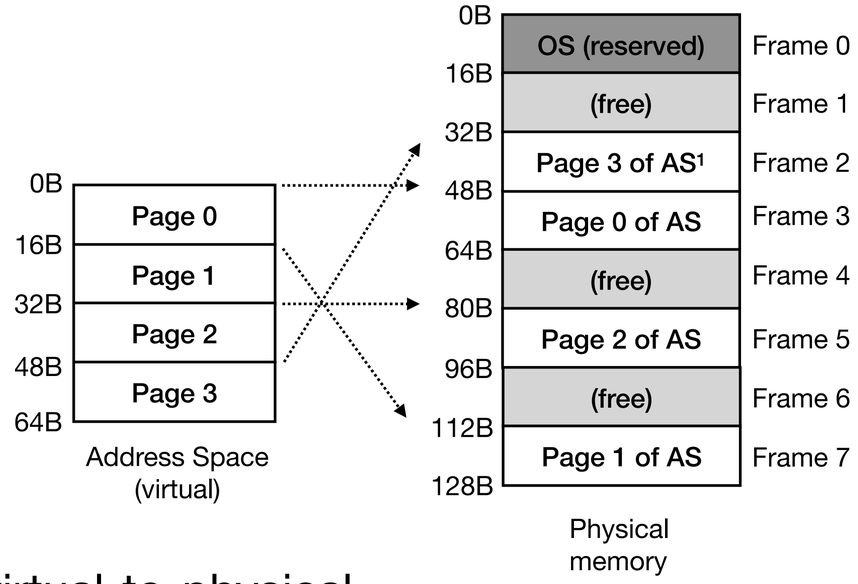
# Paging

## Pages and frames

| |
|---|
| VPN 0 → PFN 3 |
| VPN 1 → PFN 7 |
| VPN 2 → PFN 5 |
| VPN 3 → PFN 2 |

Page Table of Process
Translation from **Virtual Page Number (VPN)** to **Physical Frame Numbers (PFN)**

- The virtual address space is divided into fixed-size virtual **pages**
  - Physical memory is divided into fixed-size page **frames**
  - Each physical page **frame** contains a single virtual **page** (i.e., avoids external fragmentation)
- Advantages
  - **Simplicity** of free-space management
  - **Flexibility** in supporting the address space abstraction
- **Page Table** is the data structure (stored in main memory) that maps VPNs to PFNs
  - The **Translation Look-aside Buffer (TLB)** is a hardware cache of virtual-to-physical address translations (avoids extra memory access to the page table)
  - Linear Page Table representations consume significant main memory for large virtual address spaces

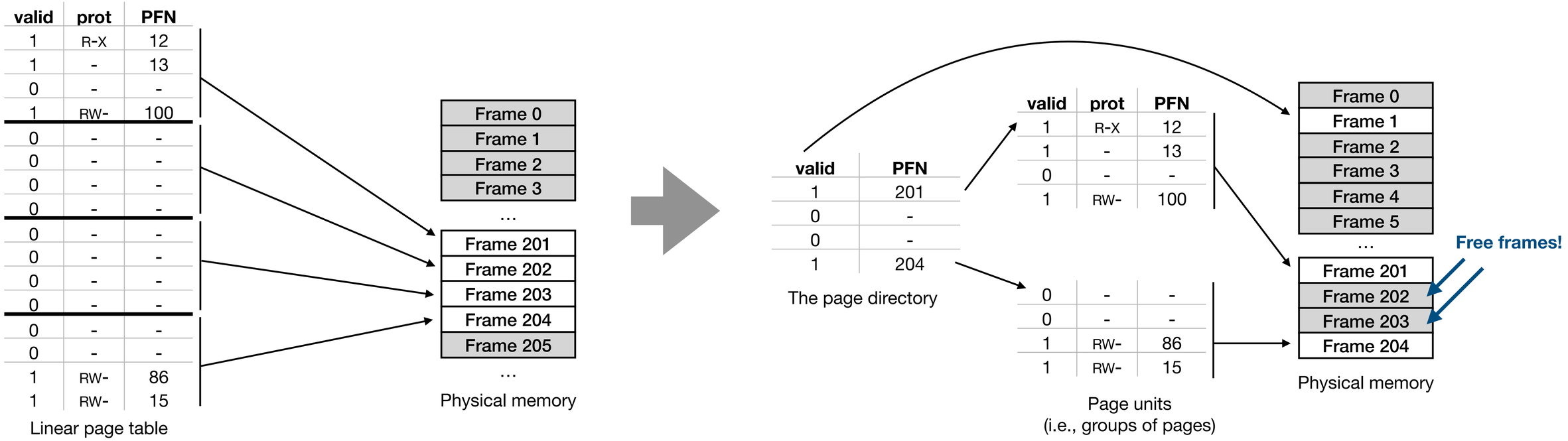



¹Address Space (AS)

# Page Table Designs

## Multi-level page tables

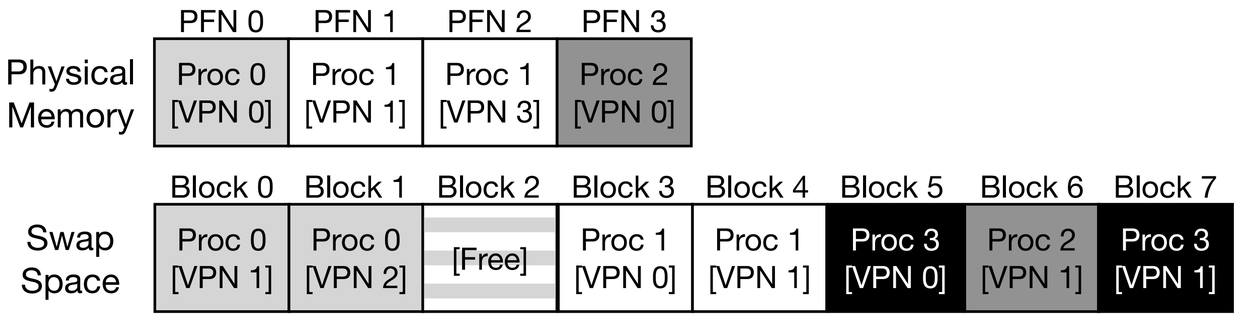
- Chop up the linear page table into **several page-sized units**
  - **Each unit** is **only allocated** at physical memory (i.e., at a given frame) if it contains **at least one valid PTE**
- A new structure, called **page directory**, is responsible for indexing the **page units** (i.e., it keeps the unit's PFN)
  - It contains a **Page Directory Entry (PDE)** per indexed unit. Each PDE has a **valid bit** and a **PFN**
  - **Example:** Assuming a page size of 16 bytes and PTEs with 4 bytes



# Supporting Larger Address Spaces

## The swap space and page faults

- What if the OS saves (**swaps**) portions of the address space (e.g., **pages**) that are not in great demand into another location with higher capacity, such as the disk device?
  - This location is called the **swap space**.
- The act of accessing a page that is not in memory is referred to as a **page fault**
  - Under a page fault, the OS must be invoked to run a **page-fault handler** (i.e., to bring the page from disk into memory)
  - What if the main memory is full?



# Page Replacement Policies

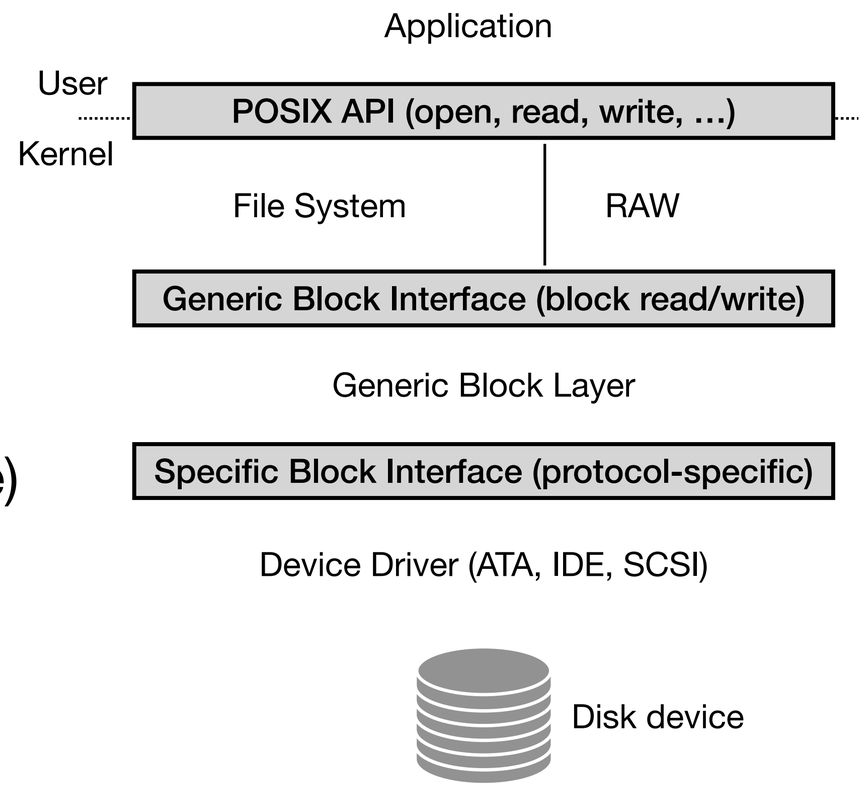
## Comparison

- Some policies
  - **First-in First-out (FIFO)** - Replace the page that has been in memory for a longer time
  - **Random** - Replace a random page
  - **Least Frequently Used (LFU)** - replace the least-frequently used page
  - **Least Recently Used (LRU)** - replace the least-recently used page
- FIFO, Random, and LRU perform similarly **when programs do not exhibit any locality property**.
- When programs exhibit **high locality**, **LRU performs better.** For instance, when a large percentage of requests (e.g., 80%) are repeatedly done to a small set of pages (e.g., 20%)
- When programs **sequentially scan large portions of memory** (e.g., a very large array) and the cache size is smaller than the number of pages being scanned, FIFO and LRU perform poorly

# Fitting Into The OS Storage Stack

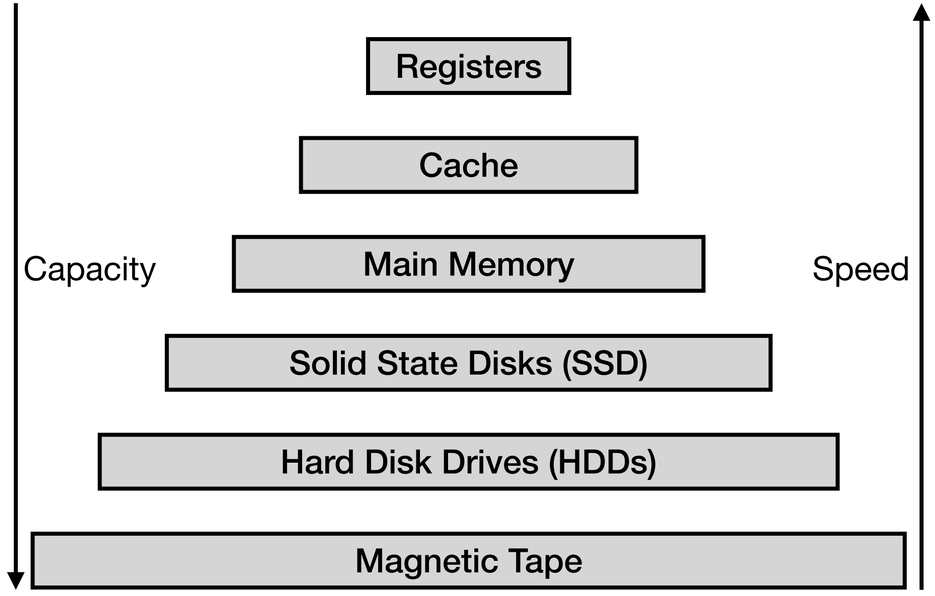
## Device driver

- I/O devices support different interfaces (e.g., ATA, IDE, SCSI, PCIe). How can the OS abstract these interfaces?
  - Solution: the **device driver abstraction**
- Example: A simplified Linux File System stack
  - **Applications** access a file system or a RAW block device, exposing the POSIX API
  - The **file system** (and applications using a raw block device) only see the generic interface exposed by the block layer (i.e., block read and write requests)
  - The **block layer** routes requests to the right **device driver**, which knows how to translate these to protocol-specific writes and reads at the I/O device



# Memory Hierarchy
## Revisited

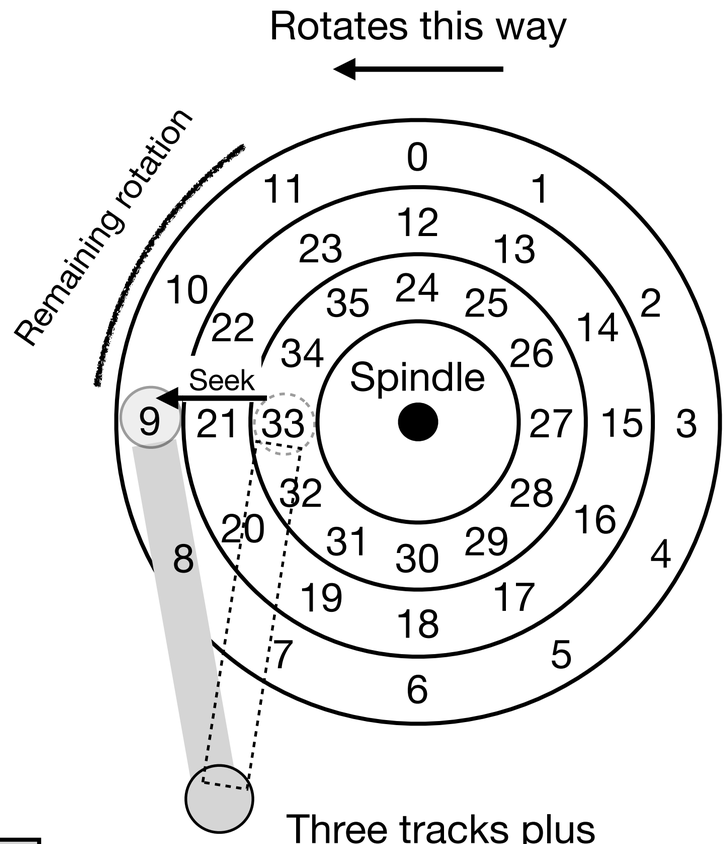
- CPU registers, caches and main memory have the fastest access times (in this order) but are **volatile**
- **Persistent** devices such as SSDs, HDDs and Tapes have slower access speeds (in this order)
- **Trade-offs:**
  - As speed increases, the cost per byte is also higher
  - To make systems affordable (and due to the laws of physics), the amount of available **fast memory** is smaller than the amount of slower **capacity** memory



# Hard Disk Drives (HDDs)

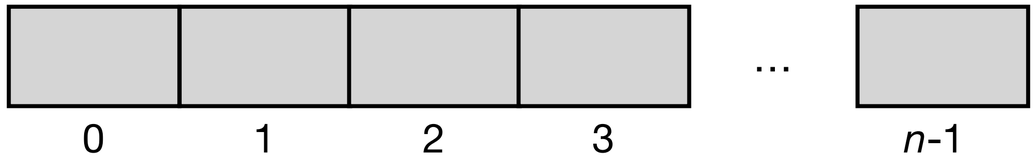
## Logical view and geometry

- Composed of **sectors** (typically 512 bytes) that can be read or written
  - A disk can be logically seen as an array of sectors (0 to *n*-1)
- Geometry-wise, data is encoded on each surface of a **platter** in concentric circles (**tracks**), each containing multiple **sectors**
  - The rate for modern drives is in the 7200 to 15000 **rotations per minute (RPM)**
- The I/O time to read/write a given sector depends on
  - the time to **move the disk arm (seek)** to the right track
  - the time to **place the disk head (rotate)** under the right sector
  - the time to **read/write (transfer)** the sector




Three tracks plus disk head and arm serving a request to **sector 11**

Logical representation of a disk (an array of sectors 0 to *n*-1)



# Scheduling Policies for HDDs

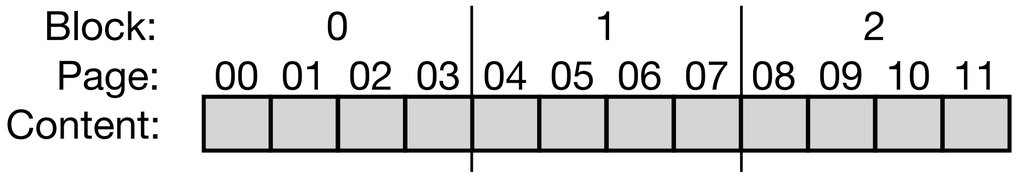
## Disk scheduler

- In older systems, disk scheduling was handled by the OS's disk scheduler
  - **SSTF**: Serve first the requests for the nearest track (**shortest seek time**)
  - **SCAN/C-SCAN**: **Sweep the disk** from inner to outer tracks (or the other way around)
  - **SPTF**: Consider **rotational** and **seek** time to choose the shortest positioning time
- In modern systems, the OS picks a few requests and lets the disk controller decide the best way to schedule them (e.g., by following SPTF)
  - The OS still implements important optimizations such as **I/O merging**
    *i.e.*, requests to contiguous sectors are merged before being sent to disk

# Solid State Drives (SSDs)

## Brief overview

- SSDs have no mechanical of moving parts, as in HDDs. Bits are stored on transistors
  - No arm seek time or head rotation time! No mechanical errors (e.g., head crashes)
- The disk is organized into flash chips, each containing two different-sized units
  - **Blocks** (also called **erasure blocks**), which are typically of size 128KB or 256KB
  - Inside each block, there are multiple **pages**[1], each with a few KBs in size (e.g., 4KB)
- Flash chips allow **reading single pages very efficiently**, regardless of their position
  - **Good for random read workloads** (unlike HDDs)!



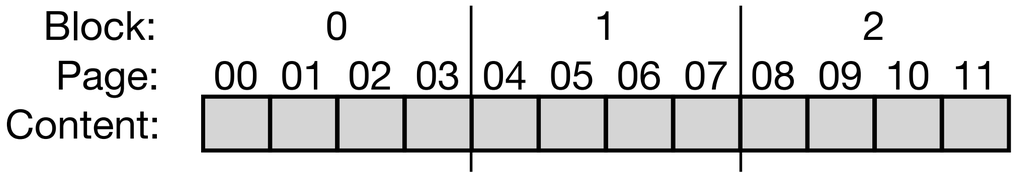

A simple flash chip with 3 blocks, each with 4 pages

[1] Do not confuse the term page here with memory pages. CS people tend to use the same term, i.e., page, block, …, for different contexts

# Solid State Drives (SSDs)

## Some interesting characteristics and challenges

- Before writing/updating a page, the **full block** containing that page must be **erased**
  - **Expensive** operation **performance-wise** (i.e., the content of valid pages must be copied somewhere else)
- Once a block has been erased, the corresponding pages can be **programmed** (i.e., written)
  - More efficient (time-wise) than erasing a block, but less efficient than reading a page
- Erasing a block also **wears out** the lifespan of the SSD drive
  - Each block can only be erased and programmed a certain number of times (e.g., 10000 times)
  - **Wear-leveling algorithms** are used to spread erase/program operations across all the blocks of the device

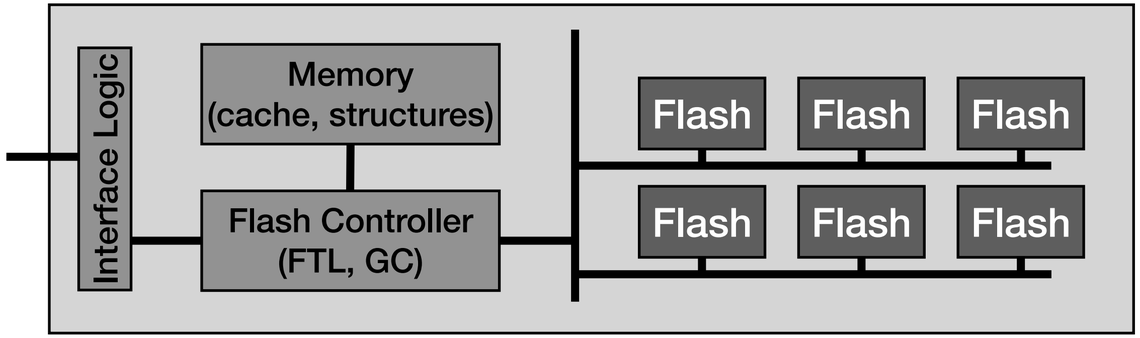



A simple flash chip with 3 blocks, each with 4 pages

# Solid State Drives (SSDs)

## Providing the disk abstraction

- How are multiple flash chips provided as a traditional storage device?
  - The SSD device offers a standard block (sector) interface atop the flash chips
- The **Flash Translation Layer (FTL)** maps **logical addresses** (device interface) to the corresponding **physical blocks and pages** (flash chips)
- The **Garbage Collector (GC)** tracks **unused pages** and **erases blocks**
- **Data structures** for the FTL and GC, along with **data caches**, are kept on volatile memory (SRAM) at the device

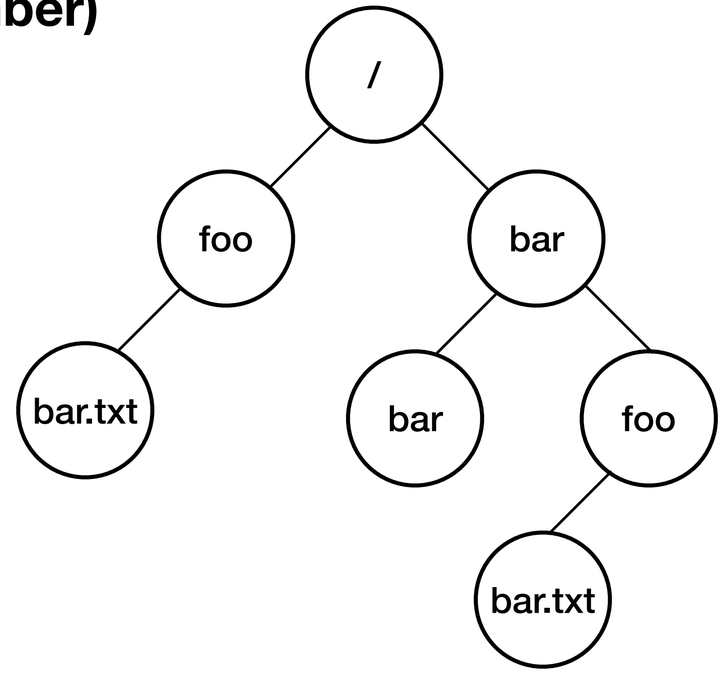



Logical Diagram of a Flash-based SSD

# File System Abstractions

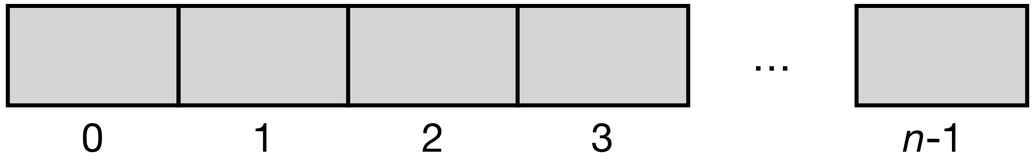
## The file and directory

- **File:** a linear array of bytes that can be read and written
  - Typically, users are only aware of the **high-level user-readable name** of a file (e.g., foo.txt)
  - Uniquely identified by a **low-level** name, the **inode number (i-number)**
- **Directory:** a container of files and other directories (also identified by a **low-level i-number**)
- A directory contains a list of pairs, each mapping a **user-readable name** to its **i-number**
  - **Example:** assuming the file *bar.txt* has i-number *10*, the directory *foo* where the file resides has the pair (*bar.txt*, *10*)




Directory tree

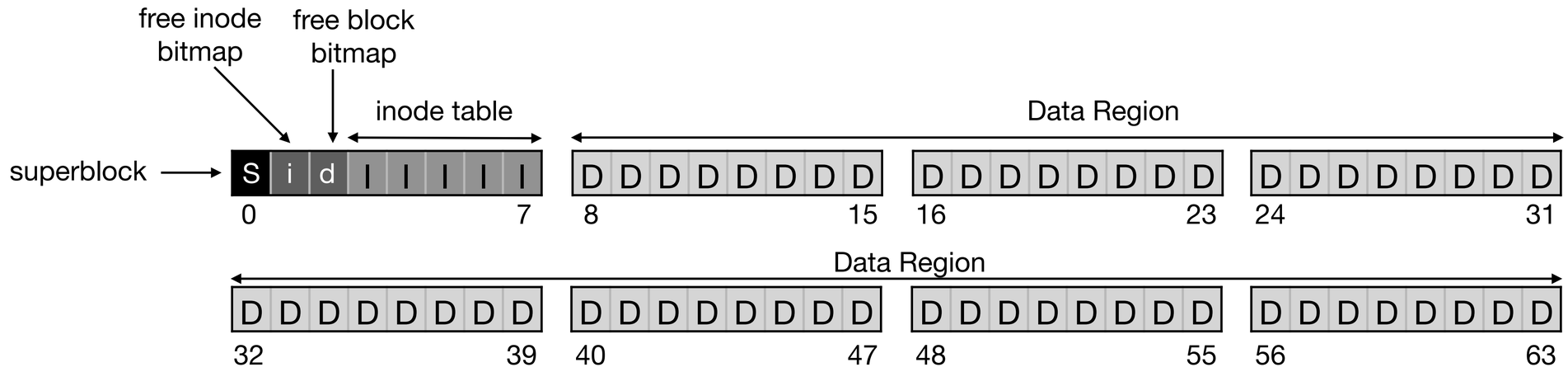
Representation of a file
(an array of bytes 0 to *n*-1)



# Very Simple File System

## Overall organization

- The **data region** holds data from the user and applications (i.e., the content of files or directories)
- The **metadata region** holds meta-information about files, directories, and the file system
  - An **inode table** holds the meta-information of each file/directory in a different **inode**
  - Two simple **bitmaps** represent whether a given inode/block is used (1) or free (0)
  - The **superblock** contains information about the FS organization (e.g., the number of inodes and data blocks, where the bitmaps and inode table begin, …)




A really small disk with 64 blocks of 4KB

# The inode

## Simplified version of an ext2 inode

| Size | Name | What is this inode field for? |
|---|---|---|
| 2 | MODE | CAN THIS FILE BE READ/WRITTEN/EXECUTED? |
| 2 | UID | WHO OWNS THIS FILE? |
| 4 | SIZE | HOW MANY BYTES ARE IN THIS FILE? |
| 4 | TIME | WHAT TIME WAS THIS FILE LAST ACCESSED? |
| 4 | CTIME | WHAT TIME WAS THIS FILE CREATED? |
| 4 | MTIME | WHAT TIME WAS THIS FILE LAST MODIFIED? |
| 4 | DTIME | WHAT TIME WAS THIS INODE DELETED? |
| 2 | GID | WHICH GROUP DOES THIS FILE BELONG TO? |
| 2 | LINKS_COUNT | HOW MANY HARD LINKS ARE THERE TO THIS FILE? |
| 4 | BLOCKS | HOW MANY BLOCKS HAVE BEEN ALLOCATED TO THIS FILE? |
| 4 | FLAGS | HOW SHOULD EXT2 USE THIS INODE? |
| 60 | BLOCK | A SET OF DISK POINTERS (15 TOTAL) |
| 4 | FILE_ACL | A NEW PERMISSIONS MODEL BEYOND MODE BITS |
| 4 | DIR_ACL | CALLED ACCESS CONTROL LISTS |

# Optimizing I/O performance

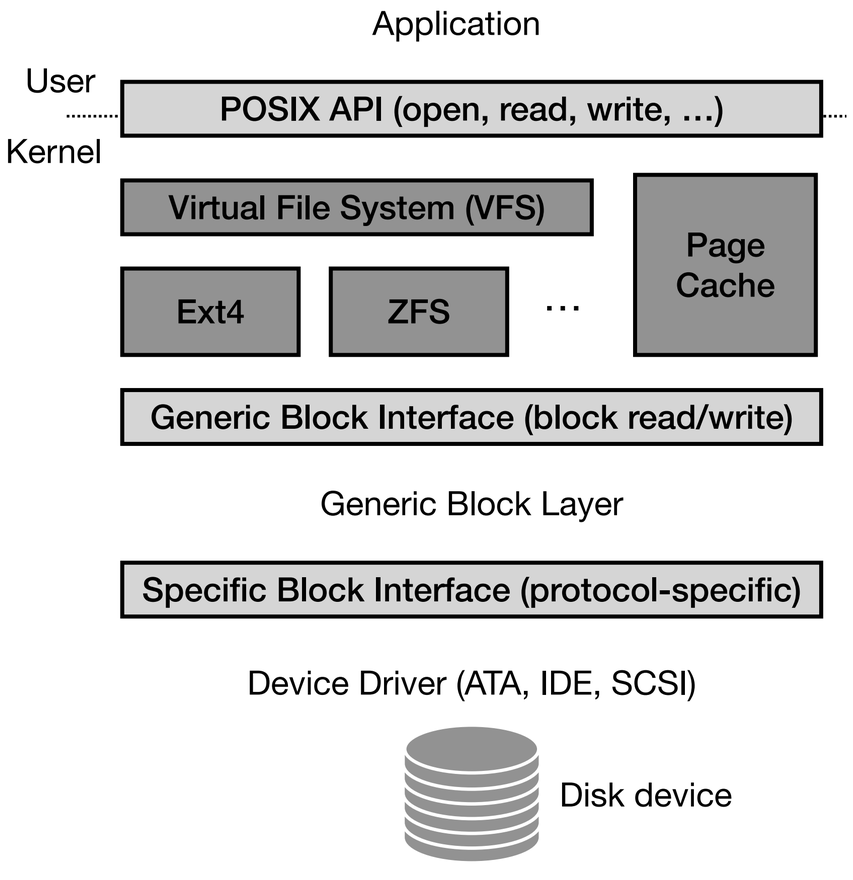
## Caching and buffering

- Opening, reading, and writing to files require several disk operations (inode/data lookups)!
  - Given the large amount of disk I/O (and since disks are slow), file systems aggressively use RAM
- Modern FSs employ a **dynamic partitioning** approach that integrates virtual memory pages and file system pages into a **unified page cache**
- Updates (writes) to inodes and data blocks can also be **buffered**
  - Opportunity to **batch** updates
  - **Problem:** If the system fails (e.g., due to a power loss) buffered data is lost!
  - **Buffering** data **improves performance** at the **cost of data durability** (a common tradeoff)

# Virtual File System

## Abstractions

- Given the multiple file system implementations available, how can the OS **abstract** them to applications?
- The **Virtual File System** (VFS) is an abstract layer on top of a specific file system implementation
  - It is useful for applications to access **transparently** different local file systems (*e.g.*, ext4, ZFS) and even remote ones (*e.g.*, NFS)
- It specifies an interface with common operations (e.g., mounting, umounting, accessing files, …)
  - Concrete file system implementations must define these "methods"
  - The interaction with OS caches is also facilitated by the VFS (*e.g.*, unified page cache, inodes/directories caches)




VFS example

# More Information

## If you want to remember specific topics

- Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. **Operating Systems: Three Easy Pieces.** Arpaci-Dusseau Books, 2018.

# Questions?